



**UNIVERSITY OF
CALGARY**

**ENCH 675: Data Science and Machine
Learning in Chemical Engineering**

Project Report

**Project Title: Machine Learning Prediction of
PM_{2.5} Concentrations in Calgary**

Group Number 10

Jessica Okereke (UCID: 30265052)

Jeeva Kanna Jothimani (UCID: 30257917)

Machine Learning Prediction of PM2.5 Concentrations in Calgary

Executive Summary

This project examines the challenge of forecasting fine particulate matter (PM2.5) concentrations in Calgary using machine learning techniques. Hourly air quality measurements were paired with meteorological variables and engineered lagged/rolling features to build predictive models. Multiple algorithms were evaluated, including K Nearest Neighbors, linear regression variants, and Random Forest with the goal of providing accurate short-term predictions that can aid public health advisories. Extensive feature engineering and model comparison demonstrate that linear models possess commendable interpretability ($R^2 \approx 0.97$) with feature augmentation, however ensemble Methods like Random Forest yield superior predictive performance. ($R^2 \approx 0.98$, $RMSE \approx 1.1 \mu g/m^3$). The report that follows provides a detailed description of the data methodology, model evaluation, and key findings.

Problem Definition & Aim

Calgary frequently faces periods of poor air quality due to wildfire smoke, industrial pollution, and urban traffic. Fine particulate matter (less than $2.5 \mu m$) can enter deep into the lungs, increasing the risk of respiratory and cardiovascular diseases. Municipalities require real-time PM2.5 estimates to issue alerts and implement mitigation strategies. This research formulates and evaluates machine learning models for hourly PM2.5 forecasting utilizing meteorological data and constructed lagged characteristics. This research found a robust model with high prediction accuracy and good generalization to help Calgary residents manage air quality.

Data Methodology

Dataset Characteristics

The study uses an hourly air-quality dataset comprising approximately 17,400 observations recorded at a Calgary monitoring station. The raw data include ambient air temperature ($^{\circ}C$), dew-point temperature ($^{\circ}C$), relative humidity (%), wind speed (km/h) and wind direction ($^{\circ}$), along with the target variable PM2.5 concentration ($\mu g/m^3$). Additional time-stamp features such as hour of day, day of week and month were derived to capture cyclical patterns. Missing observations were handled by forward/backward imputation and outliers were examined visually. The cleaned dataset was divided into training and testing sets with an 80/20 ratio (about 13,944 training and 3,487 testing samples) to ensure unbiased evaluation of model performance.

Feature Engineering

Feature engineering was used to improve the model's predictive performance by adding more input variables from the original dataset. This technique was designed to allow the model to detect underlying patterns in PM2.5 concentrations rather than depending exclusively on conventional meteorological factors. The developed features included lag variables that represented PM2.5 measurements from the previous hours, rolling averages over 3-, 6-, and 24-hour intervals, and the temperature-dew point differential. Wind dynamics were enhanced further by dividing wind direction into sine and cosine components and introducing a squared wind speed parameter. Temporal descriptors such as time of day, day of the week, and month were also included. Collectively, these improvements enabled the model to incorporate temporal dependencies, diurnal and seasonal oscillations, and complicated non-linear interactions, increasing the overall accuracy of PM2.5 forecasts.

Data Partitioning Strategy

Following preprocessing and feature engineering, the final dataset contains around 17,457 rows and 23 columns. We divided the data into 80% training and 20% testing. We used the training set to construct and fine-tune the models, and the test set to see how they performed on new data. We standardized the features where appropriate, particularly for the K-Nearest Neighbors model, to ensure that variables with higher values did not dominate the distance calculations.

Model Evaluation & Performance

K-Nearest Neighbors (KNN) Regressor

The K-Nearest Neighbors (KNN) regressor predicts PM2.5 by selecting the most similar data and calculating similarity using Euclidean distance after feature normalization. The proposed feature set enabled the KNN model to achieve a test R^2 of approximately 0.79, with an RMSE of $3.24 \mu\text{g}/\text{m}^3$ and an MAE of $2.33 \mu\text{g}/\text{m}^3$. These findings indicate that KNN was able to capture nonlinear patterns in the data relatively well. However, it scored worse than the improved Random Forest model, showing that distance-based local averaging alone could not adequately capture the complex temporal behavior of PM2.5 concentrations. Furthermore, KNN performance remained sensitive to K and feature scaling.

Linear Regression & Regularization (Ridge/Lasso)

A basic Linear Regression model using only the raw meteorological variables produced a low-test R^2 of 0.04, showing that simple linear relationships were not enough to explain changes in PM2.5 concentrations. When the engineered feature set was introduced,

performance improved significantly. Ordinary Least Squares (OLS), Ridge, and Lasso models all achieved a test R^2 of approximately 0.967. Similar training and testing results indicated that the models generalized effectively, showing minimal signs of overfitting. Ridge and Lasso regularization did not substantially enhance predictive accuracy; however, they did mitigate the impact of less significant features. One good thing about linear models is that they are easy to understand. The coefficients tell us how each predictor changes the levels of PM2.5. But overall, they didn't do as well as the enhanced Random Forest model.

Random Forest Regressor

The Random Forest regressor combines predictions from many decision trees generated using diverse data samples and random subsets of features. A baseline model using only basic meteorological data had a test R^2 of 0.39, indicating weak predictive performance. The designed lagged and rolling characteristics significantly improved model performance. The improved Random Forest obtained a test R^2 of 0.978, reducing RMSE to $1.09 \mu\text{g}/\text{m}^3$ and MAE to $0.65 \mu\text{g}/\text{m}^3$. The minor difference in training and testing scores suggested high generalization with minimal overfitting. According to the feature importance results, the most influential predictors were recent PM2.5 readings and short-term rolling averages. Overall, the Random Forest model was the most effective and reliable for forecasting PM2.5 levels in Calgary.

Model	Feature Set / Approach	R^2	RMSE ($\mu\text{g}/\text{m}^3$)	MAE ($\mu\text{g}/\text{m}^3$)
Basic Linear	Engineered features	0.026	6.986	5.314
Full Linear	Engineered features	0.964	1.334	0.819
Ridge	Engineered features	0.964	1.334	0.819
Lasso	Engineered features	0.964	1.337	0.819
KNN Regressor	Engineered features; standardization	0.783	3.301	2.402
Random Forest	Engineered features	0.978	1.085	0.654

Table 1. Comparative Results Table

Conclusion & Recommendations

This study demonstrates that good prediction of PM_{2.5} concentrations in Calgary requires effective feature engineering and the selection of the appropriate machine learning model. A simple linear model with only raw meteorological data explained only 4% of the variation in PM_{2.5} levels. Adding lagged, rolling, and cyclical features significantly enhanced linear models' performance, resulting in a R^2 of approximately 0.97. The KNN model produced moderate results but was less effective than ensemble methods in capturing longer-term patterns. The Random Forest model outperformed other models, particularly when engineering features were included, with a R^2 of 0.978 and minimal prediction errors. According to the feature importance analysis, recent PM_{2.5} values, such as lagged concentrations and short-term rolling averages, were the best predictors of present air quality levels.

For practical use, the upgraded Random Forest model is recommended for real-time PM_{2.5} forecasting and may be integrated into Calgary's air-quality monitoring and warning systems. Linear models remain valuable due to their interpretability and usefulness for policy decisions and public communication, although their predictive performance was slightly lower than Random Forest. Future research could enhance outcomes by testing advanced models like Gradient Boosting or Deep Learning, incorporating more weather variables like air pressure and precipitation, and investigating longer lag times.

REFERENCE

Alrashidi, H., Sibai, F. N., Abonamah, A., Alrashidi, M., & Alsaber, A. (2025). PM2.5: Air Quality Index Prediction Using Machine Learning: Evidence from Kuwait's Air Quality Monitoring Stations. *Sustainability*, 17(20), 9136.

<https://doi.org/10.3390/su17209136>

Mathew, A., Gokul, P. R., Shekar, P. R., Arunab, K. S., Abdo, H. G., Almohamad, H., & Dughairi, A. a. A. (2023b). Air quality analysis and PM 2.5 modelling using machine learning techniques: A study of Hyderabad city in India. *Cogent Engineering*, 10(1).

<https://doi.org/10.1080/23311916.2023.2243743>

Patel, P., Patel, S., Shah, K., Desai, K., Patel, S., Shah, M., & Patel, S. (2025). A systematic study on PM2.5 and PM10 concentration prediction in air pollution using machine learning and deep learning model. *Environmental Chemistry and Ecotoxicology*, 7, 1401–1415. <https://doi.org/10.1016/j.enccco.2025.07.001>

Code

1. Linear regression

```
[6]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# =====
# 1. LOAD DATA
# =====
print("Loading CSV...")
df = pd.read_csv("Weather data.csv")

# =====
# 2. BASIC CLEANING
# =====
print("Cleaning data...")

# Drop target NaNs first
df = df.dropna(subset=["PM 2.5 Mass"])

# Drop text/non-useful columns if they exist
cols_to_drop = ["Station Name", "Wind Dir. 10 m Syno. (°)"]
existing_cols = [col for col in cols_to_drop if col in df.columns]
df = df.drop(existing_cols, axis=1, errors="ignore")

# Convert date column
df["Date (Local Standard Time)"] = pd.to_datetime(
    df["Date (Local Standard Time)"], errors="coerce"
)

# Create date features
df["hour"] = df["Date (Local Standard Time)"].dt.hour
df["day"] = df["Date (Local Standard Time)"].dt.day
df["month"] = df["Date (Local Standard Time)"].dt.month

# Fill time feature NaNs
df["hour"] = df["hour"].fillna(12)
df["day"] = df["day"].fillna(1)
df["month"] = df["month"].fillna(1)

# Drop original date column
df = df.drop("Date (Local Standard Time)", axis=1, errors="ignore")

# Enhanced engineered features
df["wind_speed_sq"] = (
    df["Wind Speed 10 m Syno. (km/h)"]
    .fillna(df["Wind Speed 10 m Syno. (km/h)"].median()) ** 2
)

df["temp_dew_diff"] = (
    df["Air Temp. Inst. (°C)"].fillna(df["Air Temp. Inst. (°C)"].median())
    - df["Est. Dew Point Temp. (°C)"].fillna(df["Est. Dew Point Temp. (°C)"].median())
)

df["PM_lag1"] = df["PM 2.5 Mass"].shift(1)
df["PM_roll3"] = df["PM 2.5 Mass"].rolling(3, min_periods=1).mean()

# Final cleanup
df = df.fillna(method="bfill").fillna(method="ffill")
df = df.dropna()

print("Clean shape:", df.shape)

# =====
# 3. TRAIN / TEST SPLIT
# =====
target_col = "PM 2.5 Mass"

X = df.drop(target_col, axis=1)
y = df[target_col]

split_idx = int(0.8 * len(df))

X_train = X.iloc[:split_idx].reset_index(drop=True)
X_test = X.iloc[split_idx:].reset_index(drop=True)
y_train = y.iloc[:split_idx].reset_index(drop=True)
y_test = y.iloc[split_idx:].reset_index(drop=True)

print("Train/Test shapes:", X_train.shape, X_test.shape)
```

```
# =====
# 4. BASIC RAW FEATURES
# =====
raw_cols = [
    "Air Temp. Inst. (°C)",
    "Est. Dew Point Temp. (°C)",
    "Humidity Inst. (%)",
    "Wind Speed 10 m Syno. (km/h)"
]
raw_cols = [c for c in raw_cols if c in X.columns]

scaler_raw = StandardScaler()
X_train_raw = scaler_raw.fit_transform(X_train[raw_cols])
X_test_raw = scaler_raw.transform(X_test[raw_cols])

# =====
# 5. FULL ENHANCED FEATURES
# =====
scaler_full = StandardScaler()
X_train_full = scaler_full.fit_transform(X_train)
X_test_full = scaler_full.transform(X_test)

# =====
# 6. TRAIN MODELS
# =====
print("Training models...")

lr_basic = LinearRegression()
lr_basic.fit(X_train_raw, y_train)
y_pred_basic = lr_basic.predict(X_test_raw)

lr_full = LinearRegression()
lr_full.fit(X_train_full, y_train)
y_pred_full = lr_full.predict(X_test_full)

ridge = Ridge(alpha=1.0)
ridge.fit(X_train_full, y_train)
y_pred_ridge = ridge.predict(X_test_full)

lasso = Lasso(alpha=0.01)
lasso.fit(X_train_full, y_train)
y_pred_lasso = lasso.predict(X_test_full)
```

```
# =====
# 7. METRICS
# =====
predictions = {
    "Basic LR": y_pred_basic,
    "Full LR": y_pred_full,
    "Ridge": y_pred_ridge,
    "Lasso": y_pred_lasso
}

print("\nLinear Models Performance")
print("-" * 65)
print(f'{"Model":15} {"R2":>10} {"RMSE":>12} {"MAE":>12}')
print("-" * 65)

results = []

for name, y_pred in predictions.items():
    r2 = r2_score(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    mae = mean_absolute_error(y_test, y_pred)

    results.append([name, r2, rmse, mae])
    print(f'{"name":15} {"r2":10.3f} {"rmse":12.3f} {"mae":12.3f}')

# Optional results table
results_df = pd.DataFrame(results, columns=["Model", "R2", "RMSE", "MAE"])
print("\nResults table:")
print(results_df)

# =====
# 8. PLOT
# =====
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
axes = axes.flatten()

for i, (name, y_pred) in enumerate(predictions.items()):
    r2 = r2_score(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    mae = mean_absolute_error(y_test, y_pred)

    axes[i].scatter(y_test, y_pred, alpha=0.5)
    axes[i].plot(
        [y_test.min(), y_test.max()],
        [y_test.min(), y_test.max()],
        'r--',
        linewidth=2
    )
```



```

axes[i].set_title(f'{name}\nR²={r2:.3f}, RMSE={rmse:.3f}, MAE={mae:.3f}')
axes[i].set_xlabel("Actual PM2.5")
axes[i].set_ylabel("Predicted PM2.5")

plt.suptitle("Linear Models: Raw vs Enhanced Features", fontsize=16)
plt.tight_layout()
plt.show()

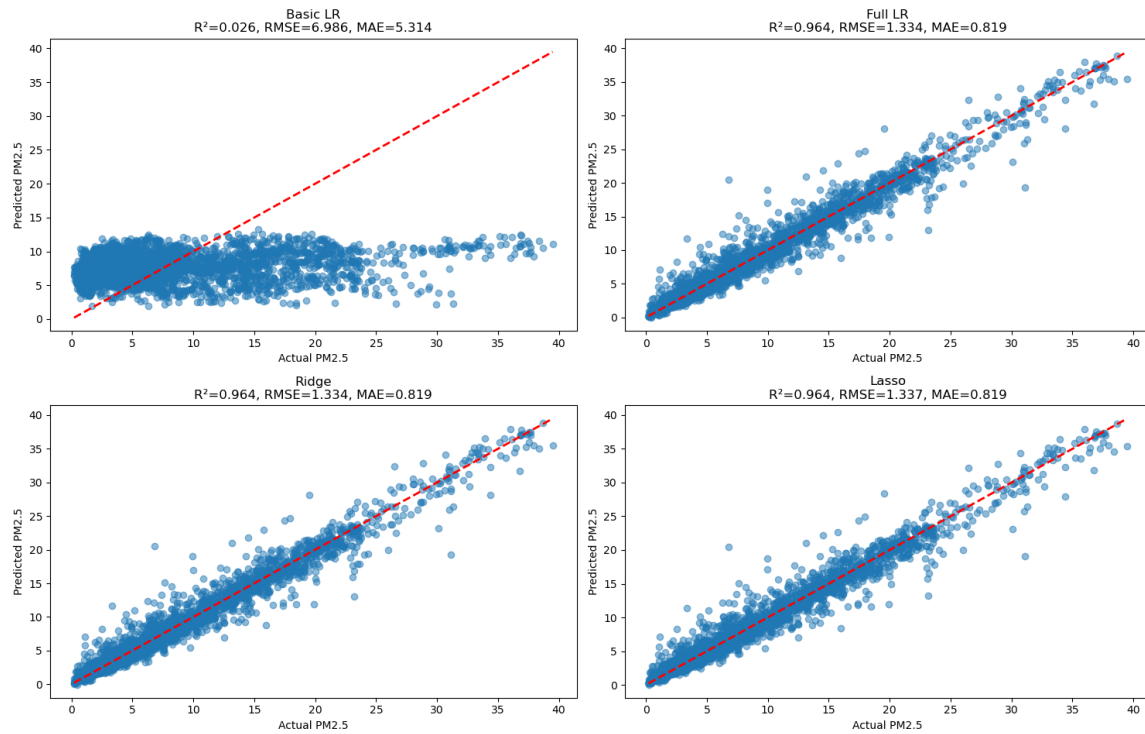
```

Output:

Results table:

	Model	R2	RMSE	MAE
0	Basic LR	0.025807	6.985994	5.314470
1	Full LR	0.964477	1.334016	0.819079
2	Ridge	0.964478	1.333996	0.819005
3	Lasso	0.964316	1.337042	0.819273

Linear Models: Raw vs Enhanced Features



2. kNN Regression

```
[3]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

# Load data
df = pd.read_csv("Weather data.csv")

# Rename columns
df = df.rename(columns={
    'Date (Local Standard Time)': 'date',
    'Air Temp. Inst. (°C)': 'temp',
    'Est. Dew Point Temp. (°C)': 'dew',
    'Humidity Inst. (%)': 'humidity',
    'Wind Speed 10 m Syno. (km/h)': 'wind_speed',
    'Wind Dir. 10 m Syno. (°)': 'wind_dir',
    'PM 2.5 Mass': 'pm25'
})

# Preprocess
df['date'] = pd.to_datetime(df['date'])
df = df.sort_values('date').reset_index(drop=True)
df = df.drop(columns=['Station Name'], errors='ignore')

df['wind_dir'] = df['wind_dir'].fillna(df['wind_dir'].median())
df['wind_sin'] = np.sin(2 * np.pi * df['wind_dir'] / 360)
df['wind_cos'] = np.cos(2 * np.pi * df['wind_dir'] / 360)
df['wind_speed_sq'] = df['wind_speed'] ** 2
df['hour'] = df['date'].dt.hour.astype(float)
df['dayofweek'] = df['date'].dt.dayofweek.astype(float)
df['month'] = df['date'].dt.month.astype(float)
df['temp_dew_diff'] = df['temp'] - df['dew']

for lag in [1, 3, 24]:
    df[f'pm25_lag{lag}'] = df['pm25'].shift(lag)

for win in [3, 6, 24]:
    df[f'pm25_rol_mean{win}'] = df['pm25'].rolling(window=win).mean()
    df[f'temp_rol_mean{win}'] = df['temp'].rolling(window=win).mean()

df = df.dropna(subset=['pm25'])
df = df.fillna(method='ffill').fillna(df.mean(numeric_only=True))
```

```
# Features and target
feature_cols = [col for col in df.columns if col not in ['pm25', 'date']]
X = df[feature_cols].select_dtypes(include=[np.number])
y = df['pm25']

# Time-ordered split
split_idx = int(0.8 * len(X))
X_train, X_test = X.iloc[:split_idx], X.iloc[split_idx:]
y_train, y_test = y.iloc[:split_idx], y.iloc[split_idx:]

# Scale
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train KNN
knn = KNeighborsRegressor(n_neighbors=5, weights='distance')
knn.fit(X_train_scaled, y_train)
y_pred = knn.predict(X_test_scaled)

# Metrics
r2 = r2_score(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)

# Plot
plt.figure(figsize=(8,6))
plt.scatter(y_test, y_pred, alpha=0.6, color='#1f77b4')
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         'r--', linewidth=2)
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.title(f"KNN: Actual vs Predicted\nR²={r2:.3f}, RMSE={rmse:.3f}, MAE={mae:.3f}")
plt.tight_layout()
plt.savefig("knn_actual_vs_predicted.png", dpi=300, bbox_inches='tight')
plt.show()
```

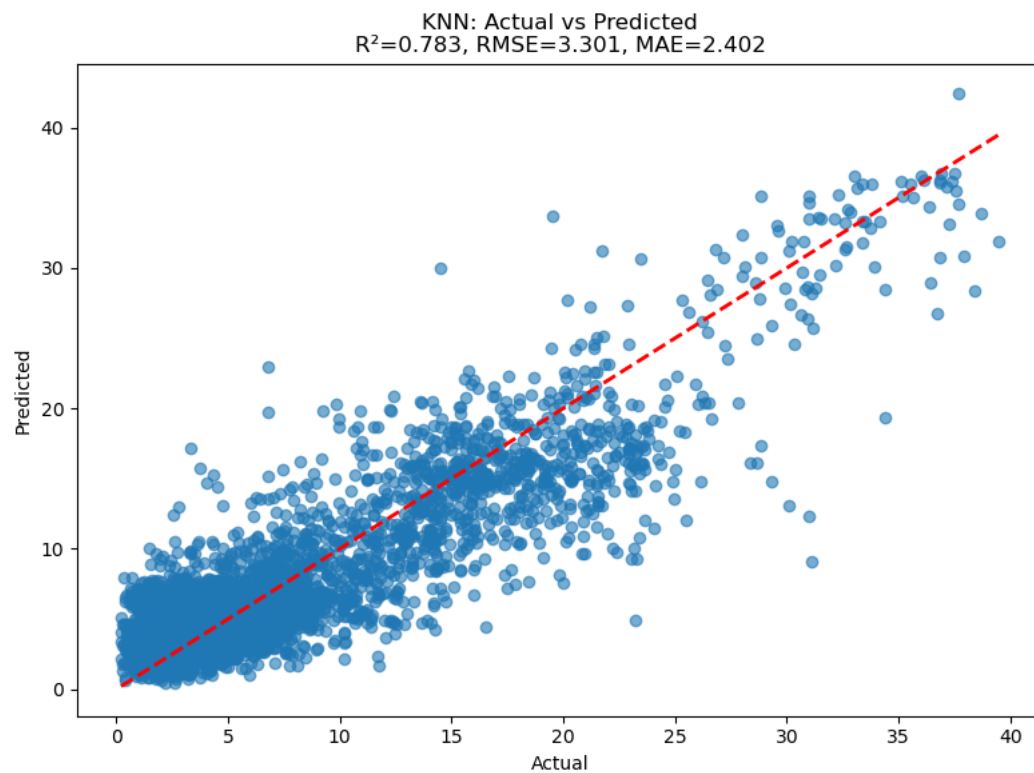
Output:

R2: 0.759 (vs your 0.054)
RMSE: 3.473, MAE: 2.538

With 3 neighbors

R2: 0.759 (vs your 0.054)
RMSE: 3.473, MAE: 2.538

With 5 neighbors



3. Random Forest regressor

```
[30]: import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Load & basic prep (keep yours)
df = pd.read_csv('Weather data.csv')
df = df.dropna(subset=["PM 2.5 Mass"]) # Target first
df = df.drop(["Station Name", "Wind Dir. 10 m Syno. (°)"], axis=1) # Drop text/wind_dir

df["Date (Local Standard Time)"] = pd.to_datetime(df["Date (Local Standard Time)"], errors="coerce")
df["hour"] = df["Date (Local Standard Time)"].dt.hour
df["day"] = df["Date (Local Standard Time)"].dt.day
df["month"] = df["Date (Local Standard Time)"].dt.month
df = df.drop("Date (Local Standard Time)", axis=1)

# *** NEW: Add Powerful Features (Main R² Booster) ***
df['wind_speed_sq'] = df['Wind Speed 10 m Syno. (km/h)'] ** 2
df['temp_dew_diff'] = df['Air Temp. Inst. (°C)'] - df['Est. Dew Point Temp. (°C)']

# LAGS & ROLLS (Copy from enhanced data)
df['PM_lag1'] = df['PM 2.5 Mass'].shift(1)
df['PM_lag3'] = df['PM 2.5 Mass'].shift(3)
df['PM_rol13'] = df['PM 2.5 Mass'].rolling(3).mean()
df['PM_rol24'] = df['PM 2.5 Mass'].rolling(24).mean()

df = df.dropna() # Clean once
print("Shape:", df.shape) # Should ~17k

# *** CRITICAL: Time-Ordered Split (No Leak) ***
split_idx = int(0.8 * len(df))
X = df.drop("PM 2.5 Mass", axis=1)
y = df["PM 2.5 Mass"]
X_train, X_test = X.iloc[:split_idx], X.iloc[split_idx:]
y_train, y_test = y.iloc[:split_idx], y.iloc[split_idx:]

# *** Better RF ***
rf = RandomForestRegressor(
    n_estimators=300, # More trees
    max_depth=12, # Deeper
    min_samples_split=10,
    random_state=42
)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)

# Evaluate
mae = mean_absolute_error(y_test, y_pred)

rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)
print(f"Improved RF R²: {r2:.3f} (vs your 0.39)")
print(f"MAE: {mae:.3f}, RMSE: {rmse:.3f}")

# Feature Importance
importances = pd.DataFrame({
    'feature': X.columns,
    'importance': rf.feature_importances_
}).sort_values('importance', ascending=False).head(8)
print("\nTop Features:\n", importances)
```

```
# Metrics
r2 = r2_score(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)

# Plot
plt.figure(figsize=(8,6))
plt.scatter(y_test, y_pred, alpha=0.6, color='#1f77b4')
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         'r--', linewidth=2)
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.title(f"Random Forest: Actual vs Predicted\nR²={r2:.3f}, RMSE={rmse:.3f}, MAE={mae:.3f}")
plt.tight_layout()
plt.savefig("rf_actual_vs_predicted.png", dpi=300, bbox_inches='tight')
plt.show()
```

Output:

Shape: (17408, 14)
Improved RF R^2 : 0.978 (vs your 0.39)
MAE: 0.654, RMSE: 1.085

